

Design Of Interrupt Based MATRIX KEYPAD EMBEDDED-C DRIVER For A Microcontroller

N. ABID ALI KHAN

Embedded system project that generally involves a microcontroller unit (MCU) user interface (UI) is a common and inevitable requirement. The simple form of arriving this option of UI is with the help of a monochrome LCD, which serves as a basic output device and a matrix keypad interface for accepting inputs from the user. In fact, majority of the UI designs are with 2x16 LCD ASCII compliant display, typically JHD162A along with either 4x4 or 4x3 matrix keypads.

Here, we start with the development of a device driver for a matrix keypad, typically written in embedded-C for microcontroller. We will consider an 8-bit AT89S52

MCU (works well for any C51 MCU), which is easily available and whose architecture is familiar to many of us. We will implement a C driver for scanning a key in the polled mode first and see how to test it without implementing or coupling the output driver (LCD).

We shall then investigate the design aspects of converting this driver by binding/locking it into an asynchronous external interrupt pin (/INT0 pin) and present the implementation and testing aspects involved in achieving driver in embedded C. It is advised to consider further aspects with Keil μ Vision 5 IDE with C51 cross-compiler (that support legacy C51 microcontroller

families) installed in Windows 10 host PC along with simulation software such as Proteus 7.7 or above for validation.

The project has four parts where the source program (Project1 through Project4) of each part is written in embedded C programming language. The hex code generated through Keil μ Vision 5 IDE is used for simulation and verification using Proteus software.

Before we dig into the technicality, we need to understand the need for this conversion. Since majority of the embedded projects deal with this standard polled-loop driver for keypad, the need to convert arises in following cases:

(a) When the MCU is expected to handle the traffic over serial (typically UART) port for achieving networking. Let us consider an example of handling the data using GSM's ASCII terminal commands (AT) in the form of SMS. While the MCU is busy in handling this SMS traffic using handshaking options, if user presses any input, forcing

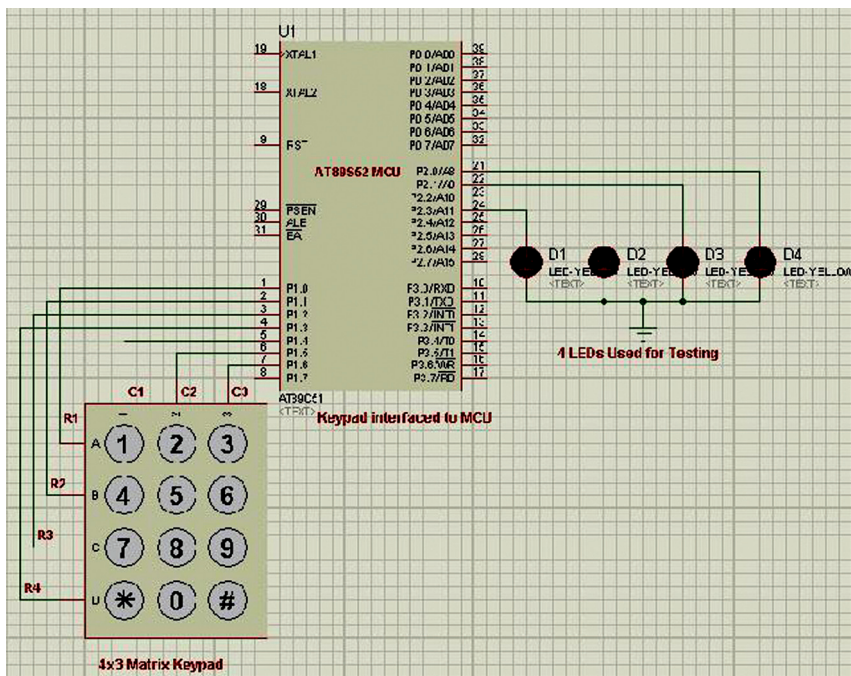


Fig. 1: Experimental setup for interfacing keypad and testing its driver

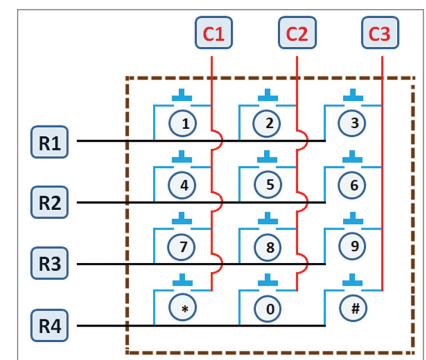


Fig. 2: Typical keypad layout diagram